

Defusing Logic Bombs in Symbolic Execution with LLM-Generated Ghost Code

DIMITRIOS STAMATIOS BOURAS, Key Lab of HCST (PKU), MoE, SCS, Peking University, China

SERGEY MECHTAEV*, Key Lab of HCST (PKU), MoE, SCS, Peking University, China

Symbolic execution is a powerful program analysis technique, but its effectiveness is fundamentally limited by solver-hostile program fragments, complex numerical reasoning, and unbounded heap structures. Recent work proposed replacing constraint solvers with large language models (LLMs) to bypass these limitations, but such approaches struggle to analyze real-world codebases, where deep execution paths require globally consistent reasoning across many interacting constraints. We present GORDIAN, a hybrid symbolic execution framework that uses LLMs selectively to generate lightweight ghost code that aids an SMT solver in handling solver-hostile code fragments, while preserving its precise, global reasoning capability. In particular, we propose three types of ghost code: (1) inversion of difficult code fragments with iterative bidirectional constraint propagation, (2) modeling via solver-friendly surrogates while preserving relevant behavior, and (3) semantic partitioning of unbounded heap spaces. We implemented GORDIAN on top of the KLEE symbolic execution engine and evaluated it on synthetic “logic bombs” capturing distinct symbolic reasoning challenges, a popular mathematical library FDLIBM, and four structured-input programs (libexpat, jq, bc and libyaml). Across benchmarks, GORDIAN improves coverage by 28.5–115.2% over traditional symbolic execution baseline and by 74.1–189.8% over LLM-based symbolic execution baselines, while reducing LLM token usage by an average of 91–96%. This highlights the practicality and effectiveness of this approach in real-world settings.

1 Introduction

Symbolic execution runs code on abstract inputs, encodes program behavior in logical constraints, and uses an SMT solver to analyze this behavior. The applicability of symbolic execution is hindered by several inherent limitations. First, because SMT solving is theoretically undecidable in general, symbolic execution often struggles with programs involving complex computations, as these translate into correspondingly complex SMT constraints. Second, symbolic execution requires precise modeling of program behavior, which becomes challenging in real-world environments, such as those involving network communication or extensive system calls, that are difficult to model accurately. Third, the number of possible program execution paths is effectively infinite, meaning symbolic execution can only explore a limited subset. The above challenges are particularly acute in programs that make extensive use of dynamic memory allocation and pointer-based data structures, where control flow depends on complex aliasing relations or heap configurations.

Recent developments in large language models (LLMs) promise to mitigate these limitations. In particular, Autobug [21], PALM [38] and ConcoLLMic [25] use LLMs in place of, or to substantially reduce reliance on, SMT solving to directly analyze the program and generate tests for difficult program parts. While this circumvents the undecidability issues of SMT solvers, these methods struggle to analyze real-world codebases when the target functionality is deep in the program, reachable only after several layers of calls and gating checks that require globally consistent choices about inputs and state. This is because LLMs struggle to maintain long-range, formally checkable dependencies across many functions, whereas SMT solvers accumulate and satisfy precise cross-procedural constraints, which enables reasoning about subtle interprocedural interactions.

We propose GORDIAN, a novel way of integrating symbolic execution with LLMs. Instead of replacing SMT solvers, GORDIAN uses an LLM only where SMT-based symbolic execution struggles.

*Sergey Mechtaev is the corresponding author.

In particular, an LLM analyzes difficult parts of the program, recognizing domain structures and identifying constraint solving strategies, and translates these into lightweight “ghost code” that plugs into the SMT-based symbolic execution workflow as domain-specific solver aids. This preserves precise, global reasoning over cross-function dependencies and subtle interactions, while alleviating hard constraints that block progress in traditional symbolic execution.

We introduce three types of LLM-generated ghost code for helping symbolic execution analyze a complex program fragment. First, we prompt an LLM to invert the difficult fragment, so that it can be executed backward to propagate concrete values from the suffix constraints back across the fragment to satisfy prefix constraints. Second, we prompt an LLM to generate a simplified surrogate that approximates complex behavior to permit efficient forward symbolic execution. Third, for program fragments operating over dynamic data structures, we prompt an LLM to generate symbolic heap topologies based on domain-specific insights to mitigate path explosion. The choice of ghost code type is context-dependent and selected by the LLM. Soundness is preserved because any inputs or states produced with ghost code are checked against the original program, so hallucinated ghost code does not introduce unsound results.

We implemented GORDIAN in the KLEE symbolic execution engine [6] and compared it against previous techniques on programs that have historically challenged symbolic execution: (1) 53 synthetic programs from the LogicBombs benchmark [39] capturing distinct symbolic-reasoning challenges; (2) 78 entry points¹ from the widely-used math library FDLlibM, which induces constraints involving undecidable theories; and (3) 19 source files from four projects (libexpat, jq, bc and libyaml) targeting structured input formats and thus operating on complex dynamic data structures. Across the benchmarks, GORDIAN improves coverage by 28.5–115.2% over the strongest traditional symbolic-execution baseline and by 74.1–189.8% over the strongest relevant LLM-based baseline while reducing the number of tokens by 91–96%. Via ablation, we confirmed that each type of ghost code contributed to the performance improvement.

In summary, the paper makes the following contributions:

- A novel enhancement of symbolic execution with LLM-generated solver-aiding ghost code.
- Three practical ghost code variants, as well as corresponding generation prompts and solver integration algorithms, to alleviate practical challenges of symbolic execution.
- A comprehensive evaluation on both synthetic and realistic benchmarks that highlights the strength of our approach in covering hard-to-reach program branches.

All code, data, and scripts are available at <https://figshare.com/s/09550df85983cbf4ae71>.

2 Overview

Ghost code [13] refers to auxiliary constructs added to a program to facilitate specification, reasoning, or verification without affecting its functional behavior. Traditionally, ghost code has been used in program verification to express invariants, ranking functions, or lemmas to automate proofs of program properties. Our work is the first that applies ghost code to enhance symbolic execution, and shows that it can be automatically generated using LLMs. This section illustrates the utility of LLM-generated ghost code for symbolically executing programs that involve non-trivial mathematics and dynamic data structures.

2.1 Propagating Constraints Through Difficult Code Fragments

Symbolic execution struggles to analyze programs with complex operations such as non-linear arithmetic or trigonometric functions, since they induce hard-to-solve constraints. Consider an implementation of the Bessel function in Figure 1a, which is a part of the function `_ieee754_j0`

¹Entry points are top-level target functions that exercise different parts of the program or library.

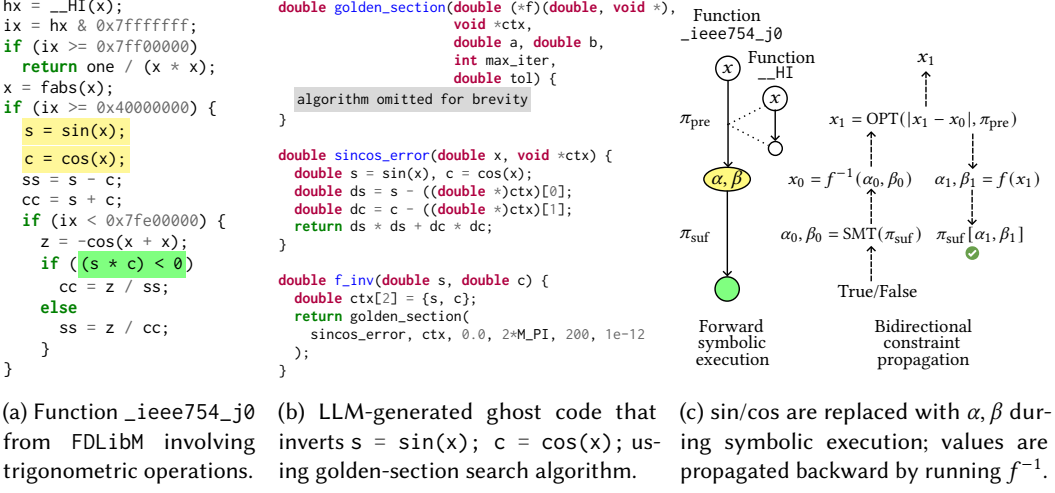


Fig. 1. GORDIAN propagates constraints through a difficult code fragment that involves trigonometric operations using a concrete execution of an LLM-generated ghost function that inverts the fragment.

from the math library FDLibM. Suppose symbolic execution aims to explore branches guarded by the condition $(s * c) < 0$ highlighted with . Even KLEE-Float [22], a version of KLEE [6] optimized for floating-point operations, fails to solve the resulting constraints, because this check depends on the output of trigonometric operations highlighted with .

As an alternative to traditional techniques, ConcoLLMic [25] replaces the SMT solver with an agentic LLM workflow to generate inputs that cover target branches. It formulates queries of the form $f(a) = b$ and then synthesizes a suitable input a using natural-language reasoning and Python tool execution. In Figure 1a, although ConcoLLMic can find a satisfying assignment for constraints originating from the difficult fragment highlighted by , this fragment itself depends on the value of x . The variable x induces a non-trivial control-flow dependency through bit-level manipulation of its IEEE-754 representation: (1) extracting the high 32 bits with `__HI(x)`, (2) masking off the sign bit with `hx & 0x7fffffff` to obtain $|x|$, and (3) checking the guard `ix >= 0x40000000`, which corresponds to $|x| \geq 2.0$. Because ConcoLLMic lacks a mechanism for global reasoning that can combine constraints arising from multiple program locations, it fails to infer and satisfy these intertwined conditions, and therefore cannot cover the target branches guarded by $(s * c) < 0$.

To address this challenge, GORDIAN operates in three steps. In step 1, it asks an LLM to identify a code fragment that is hard for an SMT solver to reason about and to generate ghost code that inverts the fragment. In this example, the LLM flags `s = sin(x); c = cos(x);`, which we refer to as f , as difficult and produces an efficient inverse procedure f^{-1} using golden-section search [18] as shown in Figure 1b. In step 2, GORDIAN executes the program on a symbolic input x , replacing the difficult fragment with fresh symbolic variables α and β that stand for the values of s and c as shown in Figure 1c. During this run it collects two constraints: (1) a prefix path condition π_{pre} from code preceding the fragment, which depends only on the input x ; and (2) a suffix path condition π_{suf} from code following the fragment, which depends on α and β .

The final step integrates the synthesized ghost code into the constraint-solving workflow via *bidirectional constraint propagation*. First, GORDIAN solves the suffix constraints π_{suf} with an SMT solver to obtain candidate values for s and c , denoted α_0 and β_0 ; assume the solver returns $(\alpha_0, \beta_0) = (10, -1)$. Next, GORDIAN propagates these values backward through the difficult fragment by concretely executing the synthesized inverse f^{-1} . Although no real x can satisfy these values

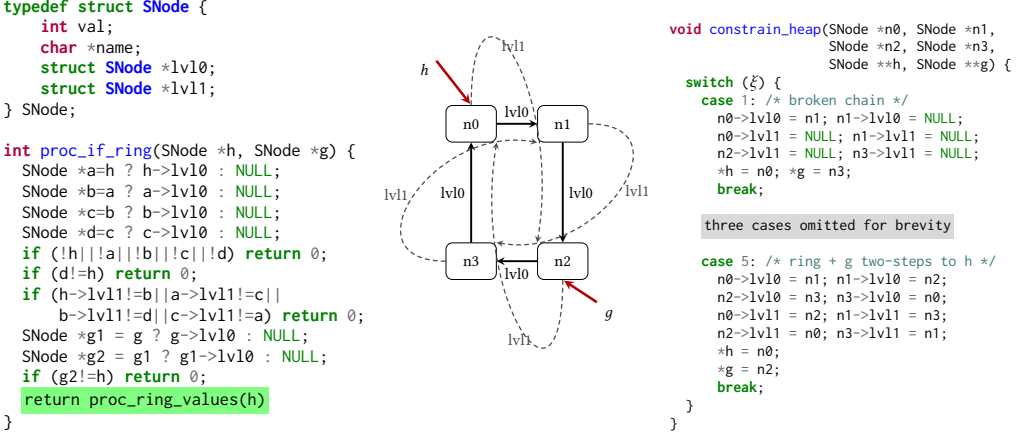


Fig. 2. GORDIAN tackles heap-configuration explosion by using LLM-generated ghost code to constrain the heap to semantically meaningful data-structure shapes while preserving symbolic contents.

simultaneously (since $\sin^2(x) + \cos^2(x) = 1$), the inverse computation yields the closest feasible input $x_0 \approx 1.67$ radians. This input violates the prefix constraint π_{pre} , which requires $|x| \geq 2.0$. GORDIAN therefore uses an optimization solver to find the closest x_1 to x_0 that satisfies π_{pre} ; in this example, $x_1 = 2.0$. It then propagates x_1 forward by executing the original fragment f , producing $(\alpha_1, \beta_1) \approx (0.909, -0.416)$, which satisfies π_{suf} . Finally, GORDIAN concretely executes the program to validate that the generated input indeed reaches the target branch, mitigating potential ghost code inaccuracies due to LLM hallucinations.

In comparison to traditional symbolic execution, GORDIAN takes advantage of LLM’s domain knowledge and code generation capabilities in the form of an inversion algorithm, for which it chose to implement a golden-section search. In comparison to LLM-based techniques such as ConcoLLMic, GORDIAN takes advantage of the SMT solver’s precise reasoning about cross-function interactions, i.e. `_ieee754_j0` and `__HI`, achieved through our bidirectional constraint propagation algorithm that plugs the generated ghost code into the solver logic. Through this synergy, GORDIAN is the only tool that provides full coverage for this function.

2.2 Constraining Heap With Semantic Topologies

A major limitation of symbolic execution in real-world testing is heap-configuration explosion caused by dynamic data structures, such as structured-input parse trees and the complex list/tree structures used in databases and file systems. Due to the complexity of these data structures and their processing code, we demonstrate how GORDIAN addresses this problem on a synthetic example of a simplified 2-level skiplist shown in Figure 2a. In this data structure, `lv10` points to the next node in the full (level-0) linked list, while `lv11` is an “express lane” pointer that skips over some nodes to speed up searches. Each node stores a key/value `val` and associated data name.

Let the function under test be `proc_if_ring` in Figure 2a. It checks whether a given skiplist forms a valid ring, as in Figure 2b, and, if so, processes the values stored in its nodes. Testing this function is challenging because reaching the call to the value-processing routine (highlighted with) requires symbolic execution to explore a prohibitive number of heap configurations, covering

many possible shapes and aliasing relationships among ten pointers. As a result, KLEE fails to reach this function within the timeout of 10 minutes. On the other hand, the LLM-based tool Cottontail [36] accounts for input structure and can generate valid data-structure shapes, but — similarly to ConcoLLMic — it cannot reconcile value constraints originating elsewhere (e.g., in `proc_ring_values`), limiting its effectiveness on large projects.

To address these limitations, GORDIAN leverages an LLM’s contextual understanding to generate ghost code, `constrain_heap`, which constrains the heap with *semantic topologies*. A semantic topology is a partially concrete, partially symbolic heap configuration that captures a meaningful use case drawn from the problem domain or program context. In this example, the LLM generates five semantic topologies — chain, non-ring, misaligned express pointers, mismatched head link, and a valid ring — shown in Figure 2c. The selected topology is determined by a fresh symbolic variable ξ . When symbolically executing the program, GORDIAN inserts a call of `constrain_heap` in front of each call of `proc_if_ring`:

```
/* n0, n1, n2, n3, h, g are initialized with symbolic memory */
constrain_heap(n0, n1, n2, n3, h, g);
int result = proc_if_ring(h, g);
```

Crucially, within each topology, only the links relevant to the intended shape are fixed; all other fields remain symbolic, including the value and name fields. This enables exploration of shape-dependent branches without over-constraining subsequent execution. As a result, GORDIAN efficiently covers all branches in the function under test, as well as the function `proc_ring_values`, within 2 seconds.

3 Background & Notation

We work in the standard Satisfiability Modulo Theories (SMT) setting. Let X denote a finite set of (typed) variables, ranged over by $\alpha, \beta, \gamma, \xi \in X$, and let there be a fixed signature (function and predicate symbols) interpreted in a background theory T (e.g., linear arithmetic). We write $\Phi_T(X)$, or just Φ for short, for the set of all well-formed formulas with free variables in X ; formulas are denoted by $\phi, \psi \in \Phi$. An assignment (or model candidate) is a map $A : X \rightarrow \text{Dom}(T)$; satisfaction is written $A \models_T \phi$. A formula ϕ is T -satisfiable if there exists A such that $A \models_T \phi$, and T -unsatisfiable otherwise. We denote by $\text{SMT}(\phi)$ an SMT-solver call that returns either SAT together with a satisfying assignment/model A , or UNSAT. For optimization, let f be a (theory) term denoting an objective value (typically numeric). We write $\text{OPT}(f, \phi)$ for an optimization-modulo-theories (OMT) call returning an assignment A^* such that $A^* \models_T \phi$ and $f(A^*)$ is optimal among all A satisfying ϕ (with MIN or MAX understood from context), or returning UNSAT if ϕ is unsatisfiable. For substitution, given a term t and variable $x \in X$, we write $\phi[x \mapsto t]$ for the capture-avoiding substitution of t for all free occurrences of x in ϕ .

Let $\mathcal{P}$ denote the set of programs, ranged over by $P, Q \in \mathcal{P}$, and let $\mathcal{S}$ denote the set of statements, ranged over by $s, t \in \mathcal{S}$; programs are (finite) sequences of statements, written $P = s_1; \dots; s_n$. We assume a set of program variables Var , denoted by v, w , distinct from the set of logical/symbolic variables X used in formulas/constraints. Procedure calls have the form `call  $p(e_1, \dots, e_n)$` for $p \in \text{Proc}$ and argument expressions $e_1, \dots, e_n$; we write $P[C \mapsto \text{call } p(e_1, \dots, e_n)]$ for replacing a designated code fragment (subprogram) C occurring in P by that call.

A (concrete) program state is a pair $\sigma = (v, h)$ where the store $v : \text{Var} \rightarrow \text{Val}$ maps program variables to concrete values and the heap $h : \text{Addr} \rightarrow \text{Cell}$ is a finite partial map from addresses to heap cells (records), with Cell determined by the language (e.g., tuples of fields); a symbolic state is a triple $\hat{\sigma} = (\hat{v}, \hat{h}, \pi)$ where the symbolic store $\hat{v} : \text{Var} \rightarrow \text{Term}(X)$ maps variables to SMT terms, the symbolic heap $\hat{h} : \text{Addr} \rightarrow \text{CellTerm}(X)$ maps addresses to (possibly symbolic) cells whose fields are SMT terms, and $\pi \in \Phi_T(X)$ is the path condition constraining the symbols. All concrete states are denoted as Σ , and symbolic states as $\hat{\Sigma}$. Satisfaction/realization is given by an

Ghost Inversion Prompt Template

Generate a function {SIGN} that inverts the original code fragment to recover the pre-state values of the variables {VARS}. If exact inversion is not possible, implement a bounded numeric search or a clamped approximation, and clearly comment on this in the output. The original function code: {CODE}

Fig. 3. A simplified fragment of GORDIAN’s prompt template for generating ghost code that inverts a difficult fragment, where SIGN is the signature of the desired function, VARS are the variables, whose values the inversion needs to compute, and CODE is the code fragment.

assignment A such that $A \models_T \pi$, yielding a concrete state $\sigma_A = (v_A, h_A)$ by evaluating all terms in $\hat{v}$ and $\hat{h}$ under A (writing $\llbracket t \rrbracket_A$ for term evaluation). A concrete program semantics is a transition relation $\rightarrow \subseteq (\mathcal{S} \times \Sigma) \times (\mathcal{S} \times \Sigma)$, and its reflexive–transitive closure is written $\rightarrow^*$. Then, a concrete execution function is defined as $\text{exec}(P, \sigma) = \sigma' \in \Sigma$ such that $\langle P, \sigma \rangle \rightarrow^* \langle _, \sigma' \rangle$.

Example 1 (Concrete execution of a simple assignment). Let P be the program $x := x + 1$ and let the initial store be σ with $\sigma(x) = 41$. Since $\llbracket x + 1 \rrbracket_\sigma = 42$, we have $\langle x := x + 1, \sigma \rangle \rightarrow \langle _, \sigma[x \mapsto 42] \rangle$. Therefore, $\text{exec}(x := x + 1, \sigma) = \sigma[x \mapsto 42]$.

A symbolic semantics is a transition relation $\Rightarrow \subseteq (\mathcal{S} \times \widehat{\Sigma}) \times (\mathcal{S} \times \widehat{\Sigma})$ that updates symbolic store/heap with symbolic expressions and updates (conjoins) the path condition, while discarding unsat branches. Then, we define symbolic execution as a function $\text{symex} : \mathcal{P} \times \widehat{\Sigma} \rightarrow 2^{\widehat{\Sigma}}$ such that

$$\text{symex}(P, \hat{\sigma}) = \{ \hat{\sigma}' \in \widehat{\Sigma} \mid \langle P, \hat{\sigma} \rangle \Rightarrow^* \langle _, \hat{\sigma}' \rangle \}.$$

Example 2 (Symbolic execution with a branch). Let the program be $P \triangleq \text{if } (x > 0) \text{ then } y := x + 1 \text{ else } y := 0$. Assume we start from the symbolic state

$$\hat{\sigma}_0 = (\hat{v}_0, \hat{h}_0, \pi_0) \quad \text{with} \quad \hat{v}_0(x) = \alpha, \quad \hat{v}_0(y) = \beta, \quad \hat{h}_0 = \emptyset, \quad \pi_0 = \top,$$

where $\alpha, \beta \in X$ are fresh logical variables. Symbolic execution explores both branches, adding the corresponding guard to the path condition $\langle P, \hat{\sigma}_0 \rangle \Rightarrow^* \langle _, \hat{\sigma}_1 \rangle$ and $\langle P, \hat{\sigma}_0 \rangle \Rightarrow^* \langle _, \hat{\sigma}_2 \rangle$, where

$$\begin{aligned} \hat{\sigma}_1 &= (\hat{v}_1, \emptyset, \pi_1), & \pi_1 &\equiv \alpha > 0, & \hat{v}_1(y) &= \alpha + 1, & \hat{v}_1(x) &= \alpha, \\ \hat{\sigma}_2 &= (\hat{v}_2, \emptyset, \pi_2), & \pi_2 &\equiv \alpha \leq 0, & \hat{v}_2(y) &= 0, & \hat{v}_2(x) &= \alpha. \end{aligned}$$

Thus, $\text{symex}(P, \hat{\sigma}_0) = \{ \hat{\sigma}_1, \hat{\sigma}_2 \}$.

4 GORDIAN

GORDIAN adds a layer on top of a traditional SMT-backed symbolic execution engine. GORDIAN first performs an exploratory KLEE run with solver timeouts reduced by 90% to identify uncovered branches, which mark hard-to-reach paths. These candidate branches are then ranked by how high they occur in the control flow and by the LOC size of the region they guard, prioritizing branches whose resolution could unlock larger still-uncovered code regions. For each branch, GORDIAN constructs the LLM input {CODE} from the branch condition, the enclosing function, preceding code that may influence it, and directly related callees, truncated to a fixed context budget. Based on this fragment and the LLM’s choice, GORDIAN generates one of three kinds of ghost code to improve constraint solving and path exploration: (1) fragment inversion, (2) surrogate models, and (3) semantic heap partitioning. The process is fully automated, not requiring manual hints or hand-crafted inversions. The ghost code is then integrated into the symbolic executor and solver as explained below.


```

int hard_func(int x) {
    return (x * x + 6) % 97;
}
int main(void) {
    int a = a;
    int b = hard_func(a);
    if (b == 42) {
        assert(a > 20);
    }
}

void f_inv(int b, int *results) {
    int count = 0;
    int t = (b - 6) % 97;
    if (t < 0) t += 97;
    for (int a = 0; a < 97; a++) {
        if ((a * a) % 97 == t) {
            results[count++] = a;
            if (count == 2) break;
        }
    }
}

```

Fig. 4. Running example to illustrate ghost inversion and bidirectional constraint propagation.

4.1 Bidirectional Constraint Propagation with Code Inversion

Given a program P in which an LLM has identified a fragment C as difficult for SMT reasoning, e.g. non-linear arithmetic, GORDIAN uses an LLM to generate a ghost inversion procedure. Intuitively, we treat C as a (partial) state transformer on a selected slice of the state — i.e., the set of program variables and heap locations that C may read or write — and synthesize code that implements an inverse of that transformer. Executing the ghost inversion in reverse then propagates constraints from the post-state back to the pre-state, allowing the solver to reason about C indirectly even when its forward semantics is not handled well by the underlying theory.

Let the sets of program variables and heap locations that a code fragment C may read/write be

$$\text{RdVar}(C), \text{WrVar}(C) \subseteq \text{Var}, \quad \text{RdLoc}(C), \text{WrLoc}(C) \subseteq \text{Addr}.$$

Then, the read/write footprints of C are defined as

$$\text{Rd}(C) \triangleq \text{RdVar}(C) \cup \text{RdLoc}(C), \quad \text{Wr}(C) \triangleq \text{WrVar}(C) \cup \text{WrLoc}(C),$$

and we denote $\sigma \upharpoonright_S$ for projecting a concrete state σ to the values of a set S of variables/heap locations. We model C as a function from the values C reads to the values it writes, $f_C : \Sigma_{\text{Rd}(C)} \rightarrow \Sigma_{\text{Wr}(C)}$:

$$f_C(\rho) = \omega \triangleq \exists \sigma. \left(\rho = \sigma \upharpoonright_{\text{Rd}(C)} \wedge \omega = \text{exec}(C, \sigma) \upharpoonright_{\text{Wr}(C)} \right),$$

Using the prompt shown in Figure 3, the LLM is requested to generate a ghost function f_{inv} that implements an inverse $f_C^{-1} : \Sigma_{\text{Wr}(C)} \rightarrow \Sigma_{\text{Rd}(C)}$ such that

$$\forall \rho \in \text{Dom}(f_C). \quad f_C^{-1}(f_C(\rho)) = \rho.$$

Operationally, this means that for any concrete state σ where both executions terminate,

$$\text{exec}(f_{\text{inv}}, \text{exec}(C, \sigma)) \upharpoonright_{\text{Rd}(C)} = \sigma \upharpoonright_{\text{Rd}(C)}.$$

In practice, many fragments cannot be fully inverted, and an LLM is instead requested to return a set of possible values that are all valid inverses, i.e. $\forall \rho \in \text{Dom}(f_C). \rho \in f_C^{-1}(f_C(\rho))$, or when no inverse values exist, the generated function is tasked to find an input value that results in the closest output state to the expected state, as shown in the example in Figure 1b.

Example 3 (Ghost inversion for a hard fragment). Consider the program fragment in Figure 4 (left). Let C be the call/statement $b := \text{hard_func}(a)$. Its read/write sets are

$$\text{RdVar}(C) = \{a\}, \quad \text{WrVar}(C) = \{b\}, \quad \text{RdLoc}(C) = \text{WrLoc}(C) = \emptyset,$$

hence $\text{Rd}(C) = \{a\}$ and $\text{Wr}(C) = \{b\}$. For this fragment, an LLM generates the function f_{inv} shown on the right of the same figure, where the output of the function is the list of values `results`.

GORDIAN replaces C with an assignment of $\text{Wr}(C)$ to fresh symbolic variables, since it is not able to directly reason about the output of C using an SMT solver. Assume $\text{Wr}(C) = \{v_1, \dots, v_k\}$, and $\beta_1, \dots, \beta_k \in X$ be fresh logical symbols w.r.t. the current symbolic state. Let the havoc statement be

$$\text{havoc}(\text{Wr}(C)) \triangleq (v_1 := \beta_1; \dots; v_k := \beta_k).$$

Then, GORDIAN transforms the program by replacing C with this havoc statement:

$$P' \triangleq P[C \mapsto \text{havoc}(\text{Wr}(C))].$$

Finally, GORDIAN symbolically executes P' from an initial symbolic state $\hat{\sigma}_0 = (\hat{v}_0, \hat{h}_0, \pi_0)$, which is $\hat{h}_0 = \emptyset$, $\pi_0 \equiv \top$, and $\hat{v}_0(a) = \alpha$ (fresh $\alpha \in X$) in the running example. Let the set of final symbolic states produced by symbolic execution of the transformed program be

$$\text{symex}(P', \hat{\sigma}_0) = \{\hat{\sigma}_1^1, \dots, \hat{\sigma}_1^m\},$$

where each $\hat{\sigma}_1^i = (\hat{v}_1^i, \hat{h}_1^i, \pi_1^i)$ is a (feasible) terminal symbolic state. To make explicit what happens before and after the replaced fragment C , we decompose each π_1^i into corresponding prefix and suffix constraints: $\pi_1^i = \pi_{\text{pre}}^i \wedge \pi_{\text{suf}}^i$, where π_{pre}^i depends only on the initial input states, while π_{suf}^i also depends on the additional symbolic variables introduced by the havoc statement². In the running example, the two final path conditions decompose as:

$$\begin{aligned} \pi_f^{(1)} \equiv (\beta = 42) \wedge (\alpha > 20) &= \underbrace{\top}_{\pi_{\text{pre}}^{(1)}} \wedge \underbrace{((\beta = 42) \wedge (\alpha > 20))}_{\pi_{\text{suf}}^{(1)}}, \\ \pi_f^{(2)} \equiv (\beta \neq 42) &= \underbrace{\top}_{\pi_{\text{pre}}^{(2)}} \wedge \underbrace{(\beta \neq 42)}_{\pi_{\text{suf}}^{(2)}}. \end{aligned}$$

For each pair of $\pi_{\text{pre}}^i, \pi_{\text{suf}}^i$, after computing the above path conditions, GORDIAN executes our bidirectional constraint propagation algorithm (Figure 5) that reconciles prefix and suffix constraints around a transformed fragment f by iteratively “meeting in the middle” using the generated inverse f^{-1} . It first solves the current suffix constraint π_{suf} to obtain an initial assignment A_{suf} ; from this it extracts proposed values for the variables written by f and runs f^{-1} to propagate them backward to a proposed pre-state σ_{pre} over the variables read by f . It then finds an assignment A_{pre} satisfying π_{pre} that is as close as possible (under a domain-specific distance discussed below) to σ_{pre} , propagates A_{pre} forward through f to get an output state σ_{out} , and symmetrically optimizes within π_{suf} to update A_{suf} so that it is close to σ_{out} . If it satisfies $\pi_{\text{pre}} \wedge \pi_{\text{suf}}$, it returns it; otherwise it repeats the loop until it reaches a fixpoint or a solver call on π_{pre} or π_{suf} returns UNSAT. Under an approximate inverse, UNSAT is operational: GORDIAN found no satisfying assignment within the current approximation and budget; the path itself may or may not be feasible.

The algorithm drives the prefix-suffix reconciliation toward a fixpoint by treating the values propagated across the cut as soft targets and enforcing consistency via optimization. Concretely, after executing f^{-1} (resp., f) we obtain a suggested valuation σ_{pre} for the pre-variables (resp., σ_{out} for the post-variables). We then choose, among all assignments satisfying the hard constraints π_{pre} (resp., π_{suf}), one that stays as close as possible to the suggested valuation.

When the interface between the two sides is a single numeric variable x , we use Z3’s optimization engine [3] and minimize an ℓ_1 distance objective:

$$A_{\text{pre}} = \arg \min_{A|=\pi_{\text{pre}}} |A(x) - \sigma_{\text{pre}}(x)|, \quad A_{\text{suf}} = \arg \min_{A|=\pi_{\text{suf}}} |A(x) - \sigma_{\text{out}}(x)|.$$

²The path condition has to be decomposed into a sequence of sub-constraints if C is executed inside a loop.

Ghost Model Prompt Template

Rewrite the original fragment so that (1) functionality (outputs, branches) is preserved, (2) the rewritten code translates to a solver-friendly SMT theory such as bitvectors or linear arithmetic. For any external library or crypto call, provide a stub implementation. The original code fragment is {CODE}.

Fig. 6. A fragment of GORDIAN’s prompt template for generating ghost model for a difficult fragment in CODE.

This makes each update the smallest possible correction needed to restore feasibility, which empirically stabilizes the iteration. When the interface consists of multiple variables $V = \{x_1, \dots, x_n\}$, optimizing a numeric distance can scale poorly. Instead, we bias updates toward changing as few variables as possible. We encode, for each $x_i \in V$, a soft equality $x_i = \hat{x}_i$ where $\hat{x}_i$ is the suggested value from σ_{pre} (or σ_{out}), and solve a *partial MaxSAT* instance with π_{pre} (or π_{suf}) as hard constraints:

$$\max \sum_{i=1}^n \mathbf{1}(A(x_i) = \hat{x}_i) \quad \text{s.t.} \quad A \models \pi_{\text{pre}} \quad (\text{resp. } \pi_{\text{suf}}).$$

In our running example, GORDIAN uses the ghost inverse f_{inv} to enumerate candidate pre-values consistent with the observed havoc value $\{a \in \text{Val} \mid a \in f_C^{-1}(42)\} = \{6, 91\}$. Finally, candidates are filtered by the remaining suffix (post-fragment) constraints over pre-variables (here, $\alpha > 20$), leaving $a = 91$ as the only value that satisfies the explored path. Figure 1 shows a more complex example.

4.2 Surrogate Models

Some difficult fragments are poor candidates for inversion: they may discard information (many-to-one), or depend on external state. In such cases, GORDIAN asks an LLM to synthesize a *surrogate model* — a solver-friendly replacement that preserves the fragment’s relevant behavior while avoiding solver-hostile theories. Let C be the difficult fragment, with read/write footprints $\text{Rd}(C)$ and $\text{Wr}(C)$, modeled as a state transformer $f_C : \Sigma_{\text{Rd}(C)} \rightarrow \Sigma_{\text{Wr}(C)}$. Using the prompt template in Figure 6, GORDIAN queries an LLM to generate a ghost fragment C_{sur} implementing a surrogate transformer $g_C : \Sigma_{\text{Rd}(C)} \rightarrow \Sigma_{\text{Wr}(C)}$, which approximates f_C while yielding solver-friendly constraints. The program is rewritten as:

$$P' \triangleq P[C \mapsto C_{\text{sur}}], \quad \text{syntex}(P', \hat{\sigma}_0) = \{\hat{\sigma}_1^1, \dots, \hat{\sigma}_1^m\}.$$

Each terminal symbolic state $\hat{\sigma}_1^i = (\hat{v}_1^i, \hat{h}_1^i, \pi_1^i)$ induces a simplified path condition π_1^i corresponding to the original program P . Because g_C may be approximate, GORDIAN treats it as a heuristic aid rather than a trusted semantic replacement. Any concrete input produced by solving constraints from P' is validated by re-executing the original program P to ensure the intended path is feasible in

Input: Prefix constraint π_{pre} , suffix constraint π_{suf} , difficult fragment f , synthesized inverse f^{-1}

Output: A model/assignment A or UNSAT

```

 $\pi \leftarrow \pi_{\text{pre}} \wedge \pi_{\text{suf}};$ 
 $\text{target}_{\text{suf}} \leftarrow \text{vars}(\pi_{\text{suf}});$ 
 $\text{target}_{\text{pre}} \leftarrow \text{vars}(\pi_{\text{pre}});$ 
 $A_{\text{suf}} \leftarrow \text{SMT}(\pi_{\text{suf}});$ 
if  $A_{\text{suf}} = \perp$  then
  | return UNSAT
end

while true do
   $\sigma_{\text{pre}} \leftarrow \text{exec}(f^{-1}, A_{\text{suf}} \upharpoonright_{\text{Wr}(f)});$ 
   $A_{\text{pre}} \leftarrow \text{OPT}(\text{dist}(\text{target}_{\text{pre}}, \sigma_{\text{pre}}), \pi_{\text{pre}});$ 
  if  $A_{\text{pre}} = \perp$  then
    | return UNSAT
  end
   $\sigma_{\text{out}} \leftarrow \text{exec}(f, A_{\text{pre}} \upharpoonright_{\text{Rd}(f)});$ 
   $A_{\text{suf}} \leftarrow \text{OPT}(\text{dist}(\text{target}_{\text{suf}}, \sigma_{\text{out}}), \pi_{\text{suf}});$ 
  if  $A_{\text{suf}} = \perp$  then
    | return UNSAT
  end
  if  $A_{\text{suf}} \models \pi_{\text{pre}} \wedge \pi_{\text{suf}}$  then
    | return  $A_{\text{suf}}$ 
  end
end

```

end

Fig. 5. Bidirectional constraint propagation.

Ghost Topology Builder Prompt Template

Identify a small number of distinct heap or pointer configurations that drive different control-flow outcomes. Generate a function {SIGN} that builds representative memory shapes and pointer configurations. Leave concrete values in the data structures symbolic. Code context: {CODE}

Fig. 7. A fragment of GORDIAN’s prompt template for generating ghost code that constrains heap, where SIGN is the expected function signature, CODE is a heap manipulating function, and data structure definitions.

P . This guarantees soundness of reported coverage but not completeness: approximate inversions, surrogate imprecision or early non-convergence may cause GORDIAN to miss feasible paths.

4.3 Semantic Heap Partitioning

Symbolic execution often fails on pointer-rich code because many possible aliasing patterns and heap shapes cause path explosion. GORDIAN mitigates this by generating ghost code that partitions the unbounded heap space into a small set of *semantic topologies* – partially concrete, partially symbolic heap configurations corresponding to meaningful shapes in the program’s domain. Such ghost code is generated automatically for all functions that operate on unbounded symbolic pointers.

Example 4 (List segment of length ≥ 3). Let $\text{head}, n_1, n_2 \in \text{Var}$ be program variables of pointer type, and next and val be data fields. Let X contain address-typed symbols $\alpha_0, \alpha_1, \alpha_2, \alpha_3$ and value-typed symbols $\beta_0, \beta_1, \beta_2$. We define a *semantic topology* that represents the shape

$$\alpha_0 \xrightarrow{\text{next}} \alpha_1 \xrightarrow{\text{next}} \alpha_2 \xrightarrow{\text{next}} \alpha_3$$

where $\alpha_0, \alpha_1, \alpha_2$ are the first three concrete links, node values $\beta_0, \beta_1, \beta_2$ are symbolic, and the tail pointer α_3 remains symbolic (so the list has length ≥ 3 , with an unconstrained remainder). This semantic topology is represented via the symbolic state $\hat{\sigma} = (\hat{v}, \hat{h}, \pi)$, where the first three nodes are *concretely* linked, while the tail pointer is symbolic:

$$\begin{aligned} \hat{v}(\text{head}) &= \alpha_0, & \hat{v}(n_1) &= \alpha_1, & \hat{v}(n_2) &= \alpha_2, & \hat{h}(\alpha_0) &= \langle \text{val} \mapsto \beta_0, \text{next} \mapsto \alpha_1 \rangle, \\ \hat{h}(\alpha_1) &= \langle \text{val} \mapsto \beta_1, \text{next} \mapsto \alpha_2 \rangle, & \hat{h}(\alpha_2) &= \langle \text{val} \mapsto \beta_2, \text{next} \mapsto \alpha_3 \rangle. \end{aligned}$$

The path condition π enforces that the three nodes are distinct, allocated, and that α_3 denotes an arbitrary (possibly null) tail:

$$\pi \triangleq (\alpha_0 \neq \text{null}) \wedge (\alpha_1 \neq \text{null}) \wedge (\alpha_2 \neq \text{null}) \wedge (\alpha_0 \neq \alpha_1) \wedge (\alpha_1 \neq \alpha_2) \wedge (\alpha_0 \neq \alpha_2).$$

GORDIAN adds heap-topology constraints by executing generated ghost code. The key idea is to (i) introduce a fresh selector variable ξ and (ii) conditionally constrain the symbolic heap to satisfy one of a small set of semantic topologies identified based on the definition of the data structure, and the code context. Each topology yields a solver-friendly, bounded set of constraints.

Let the fragment C be a call site $\text{call } p(r)$ where the argument r is pointer-typed and fully symbolic. Using the prompt template in Figure 7, GORDIAN asks the LLM to synthesize a ghost procedure $\text{constrain_heap}_p(r, \xi)$, where $\xi \in X$ is a fresh symbolic variable selecting among candidate heap topologies. The program is then rewritten as $P' \triangleq P[C \mapsto \text{constrain_heap}_p(r, \xi); C]$, inserting the ghost call immediately before C . Symbolic execution then proceeds on P' , where the added ghost code restricts the path condition to heaps consistent with the chosen topology; other topologies are explored by varying ξ .

Example 5 (Ghost code for the list-topology of length ≥ 3). Suppose we call a procedure foo with a fully symbolic list head call $\text{foo}(\text{head})$. We insert a ghost call right before it:

$$\text{constrain_heap}_{\text{foo}}(a0, a1, a2, a3, \xi); \text{ call } \text{foo}(\text{head}).$$

Table 1. Results on the LogicBombs benchmark. Each program contains exactly two critical paths (bomb-triggering and benign). Left: KLEE baselines. Right: LLM-based tools. Tokens are reported in millions. The total number of bombs is 53 and the total number of critical paths (CPs) is $53 \times 2 = 106$.

KLEE Baselines			LLM-Based Tools						
Solver	Bombs	CPs	Backend	GORDIAN			ConcoLLMic		
	Trig.	Cov.		Bombs	CPs	Tokens	Bombs	CPs	Tokens
STP	20	59	Claude 3.7	50	103	0.30	35	83	4.03
Z3	17	56	GPT-5	49	102	0.32	33	81	2.14
Float-Z3	21	60	DeepSeek-Chat	45	98	0.33	34	82	3.04

Below is one branch of `constrain_heapf00` implementing the topology from Example 4. The ghost code links three concrete nodes, while leaving the list tail and the values unconstrained.

Semantic heap partitioning is a heuristic: it biases exploration toward a finite set of meaningful topologies and may miss behaviors whose required topology is not generated or explored within the budget. This can yield false negatives but not false positives.

```
void constrain_heap(Node* a0, Node* a1, Node* a2, Node* a3, int xi) {
  if (xi == 0) { /* list segment of length >= 3 */
    assume(a0 != NULL);
    assume(a1 != NULL && a2 != NULL);
    assume(a0 != a1 && a1 != a2 && a0 != a2);
    assume(a0->val ==  $\beta_0$ ); assume(a0->next == a1);
    assume(a1->val ==  $\beta_1$ ); assume(a1->next == a2);
    assume(a2->val ==  $\beta_2$ ); assume(a2->next == a3);
  }
  /* else if (xi == 1) { ... other topologies ... } */
}
```

5 Evaluation

Our experiments address the following research questions:

- RQ1:** Do LLM-generated ghost code techniques in GORDIAN unlock solver-hostile paths and improve coverage compared to existing symbolic and LLM-based approaches?
- RQ2:** Does GORDIAN scale to real-world numerical and structured-input programs?
- RQ3:** How does LLM token cost of GORDIAN compare to prior LLM-based approaches?
- RQ4:** How quickly does GORDIAN accumulate coverage relative to baselines?
- RQ5:** Does GORDIAN explore hard-to-reach paths that other techniques miss?

Experimental Setup. GORDIAN is implemented on top of KLEE [6] with Z3 [11], and KLEE-Float was used on instances involving floating point numbers. All experiments were conducted on a single Linux machine running Ubuntu 24.04.3 LTS with an Intel® Core™ Ultra 7 258V CPU and 32 GiB of RAM. All symbolic execution runs were conducted inside Docker containers to provide isolation and reproducibility. Each tool configuration was allocated a fixed wall-clock budget of 36 hours, which was applied uniformly across all tools, baselines, and ablations, following standard practice in symbolic execution and software testing evaluations.

LLM Backends. All LLM inference was performed via external APIs; we used three state-of-the-art LLMs from different families: (1) **GPT-5** (version gpt5-2025-08-07) [31], (2) **Claude-3.7** (version claude-3-7-sonnet-20250219) [2], and (3) **DeepSeek-Chat** (version 3.2, release 2025-12-01) [12]. We used the sampling temperature of 0.5, a standard mid-range setting, and to enhance reproducibility, we cached all LLM responses so that they can be replayed deterministically [9].

Benchmarks. We used three benchmarks to stress different symbolic execution challenges:

- **LogicBombs** benchmark suite introduced by Xu et al. [39] consists of small, synthetic C programs embedding guarded “logic bombs”, each targeting a specific symbolic execution challenge (e.g., arithmetic reasoning, floating-point conditions, symbolic memory, and path

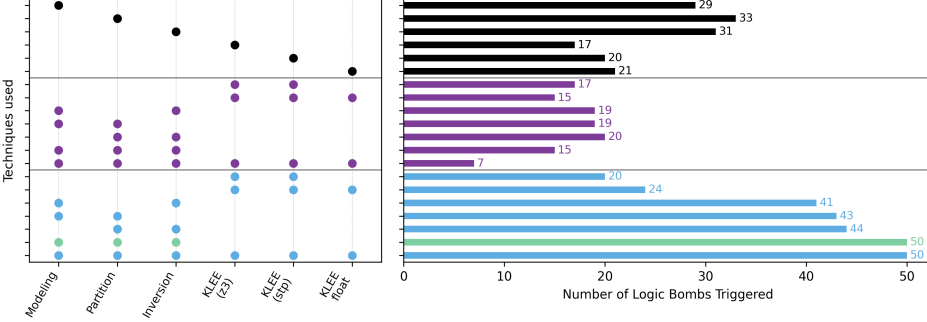


Fig. 8. Coverage across different combinations of tool components. Dots indicate enabled techniques, while bars report the number of logic bombs triggered. Black rows show individual techniques, purple rows show intersections (AND), blue rows show unions (OR), and the green row highlights the union of the three LLM-based techniques, which is equivalent to GORDIAN.

explosion). In total, it comprises 53 programs with 1,121 lines of executable code; each of them isolates a single challenge, enabling controlled analysis of which reasoning techniques are effective and how individual components of GORDIAN contribute to the overall result.

- **FDLibM** (Freely Distributable LibM) [30] is a popular math library involving complex arithmetic and floating-point guards. Our evaluation covers 78 entry points, exercising different parts of the library, totaling 5,013 lines of executable code. FDLibM contains deeply nested control flow, non-linear arithmetic, and floating-point branching that frequently leads to concretization or solver failure in conventional symbolic execution. This benchmark therefore allows us to evaluate GORDIAN’s ability to reason about inversion patterns, mathematical structure, and floating-point edge cases in real-world numerical code.
- **Structured-Input Programs**: to evaluate GORDIAN capabilities on larger, real-world codebases, we consider programs that process structured input, since such programs rely on complex dynamic data structures. We chose four widely-used programs: (1) libexpat (XML parsing, 11,828 LOC), (2) jq (JSON processing, 3,399 LOC), (3) GNU bc (POSIX expression language, 8,182 LOC), and (4) libyaml (YAML parsing, 2,760 LOC).

Baselines. We compared GORDIAN against the following baselines:

- Traditional symbolic execution: (1) **KLEE 3.0 (STP)**, using the official klee:3.0 Docker image with the STP solver, (2) **KLEE 3.0 (Z3)**, using the same image with Z3 as the backend solver, and (3) **KLEE-Float (Z3)**, using a fork of KLEE [22] designed for precise floating-point reasoning. All tools used the default settings provided by the corresponding distributions, using exactly the same options when KLEE serves as the GORDIAN backend. Thus, unless explicitly stated otherwise, GORDIAN uses the same KLEE/Z3 options as the corresponding KLEE baseline.
- **ConcoLLMic** [25] as a representative LLM-based concolic execution baseline. It first performs code instrumentation, followed by LLM-guided test generation and path exploration. ConcoLLMic’s instrumentation phase can consume much or all of the time budget before any test generation begins. To objectively assess its exploration capability, we exclude ConcoLLMic’s instrumentation time from the 36-hour budget—a choice that favors ConcoLLMic over GORDIAN. We evaluated it using the same three LLMs: GPT-5, Claude-3.7, and DeepSeek-Chat.
- **Cottontail** [36] is an LLM-based concolic executor designed for grammar-constrained input generation. Thus, we compared its performance with GORDIAN on the structured-input programs (libexpat, jq, bc, libyaml) using the same LLMs. We used JSON and XML grammars provided by the original implementation; and added grammar support for bc and YAML.

Table 2. Results on the FDLibM benchmark. Left: KLEE baselines. Right: LLM-based tools. **Avg** denotes mean per-function line coverage; **Tot** denotes overall line coverage across all functions. Tokens are in millions.

KLEE Baselines			LLM-Based Tools						
Solver	Avg	Tot	Backend	GORDIAN			ConcoLLMic		
				Avg	Tot	Tokens	Avg	Tot	Tokens
STP	62.68	70.21	Claude 3.7	91.14	83.62	0.49	40.02	31.14	16.45
Z3	63.09	68.98	GPT-5	91.67	90.23	0.54	38.65	30.62	10.53
Float-Z3	56.67	57.32	DeepSeek-Chat	91.12	83.31	0.50	37.17	28.98	8.78

5.1 RQ1: GORDIAN’s Ability To Unlock Solver-Hostile Paths

We compared GORDIAN against KLEE baselines and ConcoLLMic on LogicBombs, which is designed to exercise diverse symbolic execution challenges. Each of its 53 programs has two critical paths—a logic-bomb path and a benign one—and we report bombs triggered and critical paths covered.

Table 1 reports the results. KLEE triggers only 32.1–39.6% of bombs and covers at most 56.6% of critical paths, since solver-hostile constructs (non-linear arithmetic, floating-point conditions, complex data dependencies) leave many bomb paths unexplored. ConcoLLMic raises these to 62.3–66.0% and 76.4–78.3% by reasoning over execution semantics rather than strict constraints. GORDIAN performs best, triggering up to 94.3% of bombs and covering 97.2% of critical paths—a 138.1%/71.7% gain over the best KLEE variant and 42.9%/24.1% over the best ConcoLLMic run.

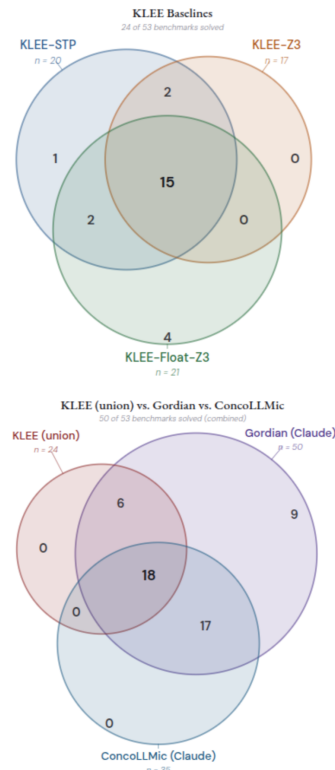


Fig. 9. Overlap in bombs KLEE, GORDIAN and ConcoLLMic trigger.

To trace this gain, Figure 8 breaks coverage down by component, using Claude for the three ghost code techniques (*Modeling*, *Inversion*, *Partition*). Each triggers a substantial but distinct subset of bombs, and their union (green row, equivalent to GORDIAN) covers nearly all of them; their intersection is small, so few bombs need all three. The KLEE variants, in contrast, largely overlap. Adding KLEE to the ghost code union yields no further coverage—GORDIAN is strictly additive over KLEE.

Figure 9 shows the overlap in bombs each approach triggers. The three KLEE solver variants agree closely, sharing 15 bombs with only marginal solver-specific differences at the margins. Both LLM-based approaches substantially extend KLEE’s reach: all share common 18 bombs, beyond which GORDIAN and ConcoLLMic jointly solve 17 more, and GORDIAN exclusively solves 9. The 50 bombs solved by GORDIAN form a strict superset of those solved by KLEE and ConcoLLMic combined, showing both that LLM-augmented techniques extend traditional symbolic execution and that GORDIAN covers all bombs reached by the other evaluated approaches on this benchmark.

RQ1: GORDIAN’s LLM-generated ghost code unlocks solver-hostile paths that are unreachable by prior approaches, triggering up to **94.3%** of logic bombs and covering **97.2%** of critical paths, strictly subsuming baseline-triggered bombs. This corresponds to a **42.9%** improvement over the strongest LLM-based baseline and a **138.1%** improvement over the best KLEE configuration.

Table 3. Line coverage (%) results on the structured-input programs. Left: KLEE baselines. Right: LLM-based tools. Avg denotes mean per-file coverage; Tot denotes overall coverage.

KLEE Baselines				LLM-Based Tools						
Bench	Solver	Avg	Tot	Backend	GORDIAN		ConcoLLMic		Cottontail	
					Avg	Tot	Avg	Tot	Avg	Tot
libexpat	Z3	20.21	14.35	Claude 3.7	46.29	38.63	11.27	4.18	11.42	11.38
	STP	18.10	12.62	GPT-5	59.72	47.75	13.04	5.11	21.86	18.30
	Float-Z3	18.40	12.89	DeepSeek-Chat	49.68	43.03	8.80	2.37	12.32	13.13
jq	Z3	34.29	28.95	Claude 3.7	46.19	45.37	9.68	8.37	30.56	29.37
	STP	35.09	29.80	GPT-5	48.69	40.98	9.75	8.56	35.34	31.74
	Float-Z3	30.81	25.34	DeepSeek-Chat	56.49	44.86	12.69	10.00	31.54	30.38
bc	Z3	50.69	40.01	Claude 3.7	70.74	59.89	15.23	6.37	52.10	39.46
	STP	54.59	44.70	GPT-5	79.15	69.79	17.13	6.87	56.36	48.12
	Float-Z3	18.01	9.84	DeepSeek-Chat	65.65	62.93	11.83	4.65	54.03	46.09
libyaml	Z3	21.03	21.17	Claude 3.7	58.03	56.51	29.74	29.17	53.5	52.43
	STP	23.67	23.74	GPT-5	63.40	65.59	27.53	26.33	44.75	43.86
	Float-Z3	15.18	13.85	DeepSeek-Chat	62.36	61.72	29.10	26.96	55.35	55.24
Overall	Z3	32.99	24.97	Claude 3.7	56.75	48.20	16.76	8.05	38.73	26.91
	STP	34.31	26.05	GPT-5	64.21	56.07	17.20	8.35	41.21	32.20
	Float-Z3	20.38	13.37	DeepSeek-Chat	59.29	51.70	15.67	6.61	40.15	30.72

5.2 RQ2: GORDIAN’s Applicability To Real-World Projects

We evaluate GORDIAN on FDLibM and the structured-input programs separately, since they involve different baselines. Table 2 reports line coverage on FDLibM, where coverage reflects each technique’s ability to explore floating-point-guarded control flow rather than reach explicit bomb branches. KLEE achieves 56.7–63.1% average and up to 70.2% total coverage; STP and Z3 perform comparably, with STP marginally ahead due to higher solver throughput. KLEE-Float-Z3 trails despite more precise floating-point reasoning, for two reasons: slow float solving limits exploration (only 21.37% as many test cases as standard KLEE), and its 9-year-old KLEE fork lacks modern performance optimizations.

ConcoLLMic, in contrast to its LogicBombs results, performs poorly on FDLibM (37.2–40.0% average, 29.0–31.1% total coverage). Even its best run with Claude-3.7 yields 55.66% less coverage than KLEE-STP despite issuing up to 16.45M tokens. This exposes a fundamental weakness of whole-path LLM reasoning on numerical code: predicting inputs that satisfy long chains of floating-point constraints is unreliable from semantic reasoning alone.

GORDIAN achieves the highest coverage on FDLibM, up to 91.7% average and 90.2% total—an average 183.5% improvement over ConcoLLMic and 22.09% over the best KLEE version, despite generating only 13.87% as many test cases. To rule out better time-budget allocation as the explanation, we combine the coverage of all three KLEE variants and all three ConcoLLMic configurations, a baseline with 6× the exploration time of a single GORDIAN run. This union reaches only 70.75% average and 77.95% total coverage, while each individual GORDIAN configuration exceeds it by 6.88–15.75% in total coverage. GORDIAN’s gains thus come from reaching solver-hostile branches that remain inaccessible even when the strengths of existing approaches are aggregated.

Table 3 reports line coverage on the structured-input benchmarks, which exercise grammar-constrained parsing and semantic validation. KLEE variants achieve modest coverage, with total coverage below 40% even for the best configuration. Z3 and STP perform comparably, while Float-Z3 trails sharply—most notably on bc, where missing inline-assembly support limits it to a single test case—and elsewhere generates under 1% as many tests as standard KLEE.

Outcome category	Attempts (%)
A1-Generation/execution failure	7
A2-Interface mismatch	8
A3-Semantically invalid inversion	22
P1-Non convergence	18
P2-Successful path trigger	45

Table 4. Mutually exclusive categories of the outcomes for 100 inversion attempts from LogicBombs.

Type	Additional Coverage (%)	Avg. Attempts
Partition	93.2	1.2
Modeling	62.1	1.4
Inversion	43.2	2.3

Table 5. Effectiveness of ghost-code strategies across LogicBombs, fdlibm, structured inputs benchmarks.

Neither ConcoLLMic nor Cottontail consistently outperforms KLEE on structured-input programs. ConcoLLMic falls below 10% total coverage—only 29.44% of KLEE-STP’s coverage on average—largely due to instrumentation failures: functions exceeding LLM output limits, and instrumentation points that cannot safely accommodate injected logging, both yielding zero coverage for the affected files. Cottontail fares better since its grammar-based generation matches structured-input programs naturally, beating the best KLEE variant by 23.61% with GPT-5; it is strongest on libyaml and bc but struggles on libexpat, where coverage requires deep semantic validation beyond syntactic correctness. Overall, end-to-end LLM-driven concolic execution remains fragile in the presence of complex cross-function dependencies.

GORDIAN outperforms all baselines on every structured-input benchmark and every backend, improving total coverage by 85.03–115.24% over the strongest KLEE, by 6.8× on average over ConcoLLMic, and by 83.36% on average over Cottontail (29.94% → 51.9% total coverage).

RQ2: GORDIAN scales to real-world numerical and structured-input programs, consistently outperforming symbolic execution and LLM-based baselines. On FDLibM, GORDIAN improves coverage by **183.5%** on average over ConcoLLMic and by **22.09%** over the strongest KLEE configuration. On structured-input programs, GORDIAN improves total coverage by up to **115.24%** over KLEE, by **83.36%** on average over Cottontail, and covers **6.8×** more lines than ConcoLLMic.

We analyze 100 randomly sampled inversion attempts from LogicBombs, excluding malformed outputs such as truncated or unparsable responses, classifying their first failure stage (Table 4). The first three categories are *artifact defects*; the last two are at least approximations of the true inverse.

(A1) Generation/execution failure (7%). Does not compile, crashes, or cannot run.

(A2) Interface mismatch (8%). Runs but output is unusable by Gordian’s refinement (missing variables, wrong type, schema violation).

(A3) Semantically invalid (22%). Correct interface, but not a valid inverse or approximation.

(P1) Non-convergence (18%). Refinement does not converge within budget. Manual inspection suggests the candidates are plausible approximations; the loop fails to close mainly due to infeasible target paths or approximation error too large for the optimization step to absorb, not inversion defects. This is the operational UNSAT case of Section 4.1.

(P2) Successful path trigger (45%). The inversion yields a concrete input validated on the original program that reaches the uncovered path.

Two rates summarize the distribution. The artifact-level success rate (defect-free inversions, A1–A3 excluded) is 63%, with 93% compiling and 85% executing. The end-to-end rate is 45%; the 18% gap is category P1, where the inversion is plausible but the refinement loop does not close—a pipeline property, not an inversion defect. Many successful inversions are themselves approximate (search-based or heuristic) rather than exact, yet propagate values consistent enough for refinement to converge. LLMs are thus reliable enough as inversion generators to aid symbolic execution.

Across FDLibM and the structured-input benchmarks, GORDIAN identifies one solver-hostile fragment per 117.5 executable lines on average, targeted with one or more of the three ghost-code

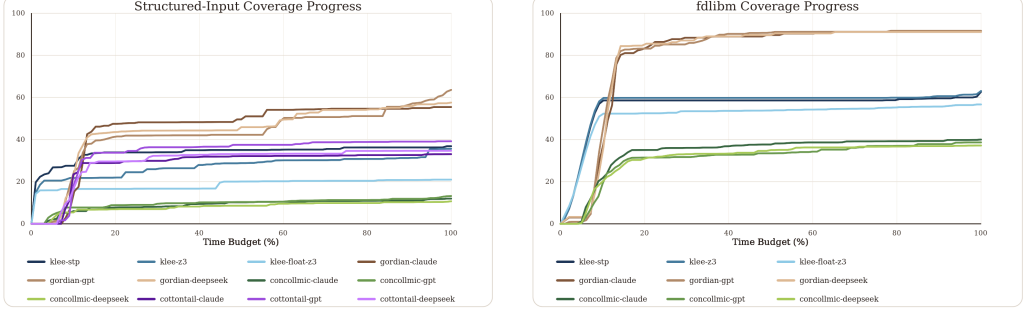


Fig. 10. Coverage progression over time for structured-input programs (left) and FDLIBM (right).

strategies (Table 5). Partitioning is the most effective, achieving the highest coverage with minimal attempts. Modeling balances generality and effectiveness. Inversion is less consistent and needs more attempts, but is crucial in solver-hostile numerical settings where other strategies fail.

5.3 RQ3: GORDIAN’s Token Efficiency

GORDIAN uses the LLM fundamentally differently from ConcoLLMic and Cottontail: it invokes the LLM only at specific solver-hostile fragments to generate a small amount of ghost code, then delegates the rest to the symbolic executor (LLM calls account for only $\sim 9.2\%$ of Gordian’s execution time). ConcoLLMic and Cottontail, by contrast, query the LLM continuously to reason about execution paths and synthesize inputs, incurring LLM cost throughout exploration.

On LogicBombs (Table 1), GORDIAN uses roughly $10\times$ fewer tokens than ConcoLLMic (0.32M vs. 3.07M per run on average). On FDLIBM (Table 2), the reduction grows to $23\times$. On the structured-input benchmarks (Table 6), ConcoLLMic consumes on average $42.69\times$ more tokens than GORDIAN, while Cottontail consumes between $5.98\times$ (GPT-5) and $17.17\times$ (Claude-3.7). GORDIAN thus achieves higher coverage at substantially lower LLM cost than existing LLM-based testing approaches.

Table 6. Total token usage (in Millions) across structured-input benchmarks.

Model	GORDIAN	ConcoLLMic	Cottontail
Claude 3.7	0.40	24.41	6.87
GPT-5	0.47	14.90	2.81
DeepSeek-Chat	0.40	14.91	5.02

RQ3: GORDIAN achieves more than an order-of-magnitude reduction in LLM token cost while achieving higher coverage. Across all 5 benchmarks, ConcoLLMic consumes **26.38 $\times$** as many tokens as GORDIAN (a **96.21%** reduction), while Cottontail consumes **11.57 $\times$** as many tokens (a **91.36%** reduction for GORDIAN).

5.4 RQ4: GORDIAN’s Coverage Progress Over Time

To assess whether GORDIAN’s gains are realized only after long runs, we measure coverage as a function of wall-clock time across all evaluated tools.

Figure 10 shows coverage versus time budget. All approaches follow a common pattern—rapid initial gains followed by a plateau—but differ in where their gains concentrate. KLEE begins exploring immediately and accrues coverage early, while LLM-based approaches incur an initial delay for preprocessing and LLM calls; once exploration starts, their growth trends resemble KLEE’s.

GORDIAN’s curves are distinctly step-wise: large early jumps correspond to successful ghost-code transformations (partitioning, modeling) that unlock previously unreachable regions, and subsequent improvements occur in discrete steps as new partitions or solver-aiding transformations



Fig. 11. Overlap of covered lines across techniques on libyaml, aggregated across all models. The largest intersection corresponds to lines covered exclusively by GORDIAN (19.2% of all covered lines), exceeding even the shared GORDIAN–Cottontail intersection (17.8%).

are introduced. GORDIAN typically surpasses both KLEE and ConcoLLMic well within the first few hours, and the bulk of its advantage over KLEE is realized before the long-budget plateau—meaning the gains are not an artifact of the 36-hour ceiling. Cottontail and ConcoLLMic, by contrast, grow more smoothly because their LLM interactions are interleaved continuously with exploration.

On FDLibM, KLEE quickly generates floating-point-heavy test cases for high initial coverage, but stagnates early as many paths hit solver-hostile constraints it cannot extend. GORDIAN continues to make progress by resolving such bottlenecks via targeted transformations. Minor spikes in the KLEE curves are a measurement artifact: coverage updates only when test cases are generated, and several paths sharing prefixes can reach new code within a short window.

RQ4: GORDIAN’s advantage emerges early rather than only at long budgets. Many of its gains over KLEE and ConcoLLMic appear within the first 20% of the 36-hour budget, while additional gains continue later through step-wise improvements unlocked by ghost-code transformations.

5.5 RQ5: GORDIAN’s Exploration of Hard-to-Reach Paths

To check whether GORDIAN’s gains correspond to genuinely new paths rather than redundant coverage, we analyze the overlap of covered lines across techniques. Coverage is a standard testing-effectiveness proxy in symbolic and concolic execution and provides more stable evaluation signals than bug-based metrics [26]; the overlap view further reveals how much coverage is shared and how much each technique contributes uniquely.

Figure 11 shows the overlap on libyaml, aggregated across LLM backends (the best-performing structured-input benchmark for LLM-based approaches). Three observations stand out: GORDIAN achieves the largest overall coverage; it contributes a substantial set of lines covered by no other approach, reaching previously unexplored behaviors; and although it overlaps significantly with structure-aware Cottontail, it still extends coverage beyond their union—its gains are not reducible to input-structure modeling alone.

RQ5: GORDIAN reaches hard coverage regions not attained by the other evaluated techniques. On `libyaml`, **19.2%** of covered lines are reached by GORDIAN alone—exceeding even its overlap with Cottontail—so its advantage cannot be explained by input-structure modeling or redundant coverage of paths reachable by other techniques.

5.6 Rerun Variability Analysis.

To quantify the impact of LLM stochasticity, we evaluate a representative subset of 16 source files (around 10% of the total), uniformly sampled across benchmarks. For each file and each of the three LLM backends, we perform three independent reruns, yielding 48 benchmark–model pairs and 144 executions in total.

We measure variability using the within-pair standard deviation across reruns. The average within-pair standard deviation is 3.08 coverage points overall, with per-model values of 2.65 (Claude), 2.36 (GPT), and 4.22 (DeepSeek), indicating moderate stochasticity at the level of individual runs.

To assess the impact on results, we approximate the standard error of average coverage over N files as $SE \approx s/\sqrt{N}$. This yields an approximate 95% stochasticity band of ± 0.68 coverage points for FDLibM (78 files; ± 0.59 for Claude, ± 0.52 for GPT, and ± 0.94 for DeepSeek) and ± 1.38 points for structured-input programs (19 files; ± 1.19 for Claude, ± 1.06 for GPT, and ± 1.90 for DeepSeek). Per-model bounds remain below ± 1 point for FDLibM and below ± 2 points for structured benchmarks. These deviations are small relative to the observed improvements (double digits coverage points over baselines), indicating that our findings are robust to LLM stochasticity.

6 Discussion

Although GORDIAN significantly outperformed baselines in the evaluation, it has several limitations. First, GORDIAN operates by injecting LLM-generated ghost code and recompiling the target program for symbolic execution. This introduces engineering overhead, especially for large systems with complex build pipelines, external dependencies, or custom build scripts. In practice, the overhead is mitigated by incremental builds: GORDIAN modifies only the harness or a small number of target/source files, so build systems such as `make` typically recompile only the affected objects and relink the executable rather than rebuilding the entire project. Nevertheless, manual integration effort may still be required for complex projects. Second, GORDIAN relies on general-purpose LLMs, which may reflect biases or implicit knowledge from their training data. In addition, LLM-generated ghost code may be incomplete or imprecise, particularly for inversion and semantic modeling. Importantly, GORDIAN treats all LLM outputs as heuristic aids: all generated test inputs are replayed on the original program, ensuring that incorrect abstractions affect efficiency rather than soundness; however, it may still miss feasible paths. Third, GORDIAN is most effective when solver-hostile behavior is localized to identifiable program fragments. Programs whose complexity arises from deeply entangled global invariants or hard-to-model environment interactions may benefit less from ghost code generation. Fourth, GORDIAN’s bidirectional constraint propagation algorithm in Section 4.1 relies on heuristic optimization, and does not guarantee convergence.

A natural direction for future work is a tighter integration of GORDIAN’s capabilities directly inside a symbolic execution engine. In our current implementation, GORDIAN operates as an external orchestrator that interacts with KLEE through repeated recompilation and execution. While effective, this architecture introduces engineering complexity and multiple points of failure. A more principled approach would embed LLM assistance natively within KLEE. Solver queries or path fragments that exceed resource limits could be detected online and selectively delegated to LLM-based reasoning modules. Once a generated model or inverse is validated by successfully triggering a previously unreachable path, it could be cached and reused across executions or

program versions. Over time, this would yield an evolving library of solver-friendly models and inverses, potentially compositional and shared across analyses.

A limitation of the bidirectional constraint propagation algorithm presented in Section 4.1 is that it assumes a single difficult fragment along any explored path. Accordingly, our implementation never attempts to invert more than one fragment at a time. This is a pragmatic choice rather than a fundamental restriction: empirically, addressing even a single difficult fragment yields substantial practical gains. In principle, however, the algorithm extends naturally to paths containing multiple difficult fragments, as follows. Suppose symbolic execution of the transformed program P' encounters a sequence of n difficult fragments $C_1, \dots, C_n$ along a single path, each equipped with an LLM-generated inverse f_i^{-1} . The accumulated path condition then decomposes into $n + 1$ contiguous segments,

$$\pi = \pi_0 \wedge \pi_1 \wedge \dots \wedge \pi_n,$$

where π_0 constrains only the original input variables (the prefix preceding C_1), and each subsequent π_i ($1 \leq i \leq n$) is the segment between C_i and C_{i+1} (with π_n the final suffix), depending on the fresh symbolic variables used in $C_1, \dots, C_i$ as well as on the original inputs.

The generalized procedure might proceed by “meeting in the middle” across all n cuts simultaneously. It first solves the bottom segment π_n to obtain an assignment A_n , then propagates values upward fragment by fragment: from A_i it extracts the values written by C_i , applies f_i^{-1} to obtain a suggested pre-state σ_i over the variables read in C_i , and checks whether σ_i together with A_i is consistent with the segment π_{i-1} . If every π_{i-1} is satisfied along the way and reaches π_0 consistently, a model for the whole path has been found and the algorithm terminates. Otherwise, at the highest segment π_{j-1} that becomes unsatisfied, the algorithm invokes the optimizing solver to compute an assignment $A_{j-1} \models \pi_{j-1}$ that is as close as possible to σ_j , and then propagates A_{j-1} downward by concretely executing $f_{j-1}, f_j, \dots, f_n$ in turn. The upward-downward traversal is then repeated from the new assignment, until either a fixpoint that satisfies $\pi_0 \wedge \dots \wedge \pi_n$ is reached, or the procedure stalls and conservatively returns UNSAT. We leave the implementation, convergence analysis, and empirical evaluation of this multi-fragment extension to future work.

7 Related Work

Symbolic and Concolic Testing. Symbolic execution [19] (e.g., KLEE [6]) explores program paths over symbolic inputs, but is limited by path explosion and solver bottlenecks, especially with complex arithmetic and hard-to-model environments [5, 21]. Concolic execution [15] improves practicality by interleaving concrete runs with symbolic reasoning; systems such as S2E [8] scale this to whole-system settings, while compilation-based engines (e.g., SymCC [32]) improve throughput via native execution and lightweight instrumentation. Prior work addresses exploration blowups from loops and recursion (e.g., MARCO [17]), whereas GORDIAN targets the complementary bottleneck: solver-hostile fragments that block progress. Environment modeling via synthesis (e.g., SE-ESOC [28]) replaces missing or external semantics with generated code; GORDIAN differs in that it targets internal program fragments whose code is present but solver-hostile, and uses ghost code not merely as a stub but as a solver aid for propagating and reconciling constraints across the original fragment. GORDIAN generalizes this idea by using LLMs to generate ghost code that (i) propagates constraints through hard fragments, (ii) introduces solver-friendly surrogates that preserve relevant behavior, and (iii) constrains pointer-rich state spaces using semantic heap topologies. For dynamic data structures, classic lazy initialization [37] incrementally materializes heap objects but remains largely shape-agnostic; GORDIAN instead injects domain-informed topological partitions to reduce search space explosion while keeping data fields symbolic.

Hybrid Fuzzing and Symbolic Execution. Compared to symbolic/concolic execution, fuzzing (e.g., AFL++ [14]) offers high throughput but typically struggles with deep, constraint-heavy branches [27]. Systems such as Driller [34] and QSYM [40] are complementary to GORDIAN: they combine fuzzing with concolic execution to improve input-space exploration, whereas GORDIAN combines LLM guidance with symbolic execution to target solver-hostile internal fragments through selective solver-aiding ghost code rather than repeated input mutation or full-path reasoning.

LLM-based Symbolic and Concolic Execution. Recent work integrates machine learning and large language models (LLMs) into symbolic execution to mitigate scalability limits caused by complex constraints and environment modeling. Early approaches focused on learned path exploration heuristics, using techniques such as Monte Carlo Tree Search [35] or learned branch prioritization policies (Learch [16]). More recent systems embed LLMs directly into symbolic or concolic execution. Solver-free approaches such as PALM [38] and AutoBug [21] rely on LLMs to generate satisfying inputs for program fragments or path-wise slices, bypassing SMT solvers at the cost of global constraint consistency and scalability on deep paths. Cottontail [36] targets structured-input programs using LLM-guided grammar-based input generation, while ConcoLLMic [25] adopts an agentic design in which LLMs summarize traces and reason about constraints. Although LLMs improve coverage by bypassing solver bottlenecks, we showed that these systems are ineffective in handling complex program dependencies. Our hybrid approach that combines LLMs with SMT solvers addresses this problem.

LLM-assisted Fuzzing and Test Generation. LLMs have also been applied to fuzzing and test generation primarily as input generators, rather than as engines for symbolic reasoning. CodaMOSA [20] uses language models to synthesize new tests when coverage plateaus, while other systems apply LLM-driven mutations to protocol and application fuzzing [24, 29]. Although effective at expanding the input space and improving shallow coverage, these approaches do not reason over execution paths or symbolic constraints and thus remain limited in reaching deep program states [27].

Program Inversion. Program inversion aims to generate an inverse program that computes inputs that lead to a given output. Previous work synthesized inverse computations using symbolic execution and solvers [1, 33], however these techniques are typically limited to small fragments or solver-friendly semantics, making them unsuitable for our use case. Just-tri-it [10] uses LLM-generated algorithm inverses to check cross-task solution consistency, which improves the quality of generated code. GORDIAN applies an LLM to invert only a small program fragment, which is easier, and uses such inverses to propagate values across execution paths. Unlike prior inversion work, GORDIAN does not require synthesizing a complete inverse program; it uses approximate fragment inverses inside a bidirectional meet-in-the-middle loop that combines LLM-generated code with SMT reasoning.

Reasoning About Programs with LLMs. Recent work has studied and improved LLMs’ ability to reason about code [7, 23]. HoarePrompt [4] structures chain-of-thought using ideas from strongest postcondition calculus, closely related to symbolic execution. AutoBug [21] improves LLM reasoning by decomposing programs into simpler components. GORDIAN is complementary to these approaches: advances in LLM reasoning can directly improve the quality of the ghost code.

8 Conclusion

This work presents GORDIAN, a hybrid symbolic execution framework that addresses key bottlenecks of SMT-backed symbolic execution using LLMs without giving up the ability to enforce globally consistent, cross-procedural constraints. Rather than replacing the solver with an LLM, GORDIAN uses the LLM selectively to generate lightweight, solver-aiding ghost code tailored to solver-hostile

fragments: inverse procedures for bidirectional constraint propagation, solver-friendly surrogate models that preserve control-relevant behavior, and semantic heap topologies that curb path explosion in pointer-rich code. Implemented on top of KLEE and evaluated across synthetic logic bombs, a real mathematical library, and large structured-input applications, GORDIAN consistently improves coverage and reduces LLM cost relative to both traditional symbolic execution and prior LLM-driven alternatives.

Data Availability

All experimental artifacts required to reproduce the results of in this paper are publicly available at: <https://figshare.com/s/09550df85983cbf4ae71>. The artifact package includes: (i) the complete implementation of GORDIAN, (ii) all baseline implementations and artifacts for KLEE, ConcoLLMic, and Cottontail, (iii) benchmark programs and harnesses for all six evaluated benchmark groups (LogicBombs, FDLiBm, GNU bc, jq, libexpat, and libyaml), (iv) generated test cases, execution logs, and intermediate artifacts, and (v) post-processed results used to produce the tables and newly added analyses in the paper.

References

- [1] Sergei M. Abramov and Robert Glück. The universal resolving algorithm: Inverse computation in a functional language. In *Mathematics of Program Construction (MPC)*, volume 1837 of *LNCS*, pages 187–212. Springer, 2000.
- [2] Anthropic. Claude 3.7 sonnet. <https://www.anthropic.com/claude>. Model version: claude-3-7-sonnet-20250219.
- [3] Nikolaj Bjørner, Anh-Dung Phan, and Lars Fleckenstein. vz-an optimizing smt solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 194–199. Springer, 2015.
- [4] Dimitrios Stamatios Bouras, Yihan Dai, Tairan Wang, Yingfei Xiong, and Sergey Mechtaev. Hoareprompt: Structural reasoning about program correctness in natural language. *arXiv preprint arXiv:2503.19599*, 2025.
- [5] Stefan Bucur, Vlad Ureche, Cristian Zamfir, and George Candea. Parallel symbolic execution for automated real-world software testing. In *Proceedings of the Sixth Conference on Computer Systems*, EuroSys '11, page 183–198, New York, NY, USA, 2011. Association for Computing Machinery.
- [6] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *OSDI*, volume 8, pages 209–224, 2008.
- [7] Junkai Chen, Zhiyuan Pan, Xing Hu, Zhenhao Li, Ge Li, and Xin Xia. Reasoning runtime behavior of a program with llm: How far are we? *arXiv preprint arXiv:2403.16437*, 2024.
- [8] Vitaly Chipounov, Volodymyr Kuznetsov, and George Candea. The s2e platform: Design, implementation, and applications. *ACM Trans. Comput. Syst.*, 30(1), February 2012.
- [9] Yihan Dai, Dimitrios Stamatios Bouras, Haoxiang Jia, and Sergey Mechtaev. Statistical independence aware caching for llm workflows, 2025.
- [10] Yihan Dai, Sijie Liang, Haotian Xu, Peichu Xie, and Sergey Mechtaev. Reducing hallucinations in llm-generated code via semantic triangulation. *arXiv preprint arXiv:2511.12288*, 2025.
- [11] Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008.
- [12] DeepSeek AI. Deepseek-v3.2 technical report. <https://www.deepseek.com/>, 2025. Release date: 2025-12-01.
- [13] Jean-Christophe Filliâtre, Léon Gondelman, and Andrei Paskevich. The spirit of ghost code. *Formal Methods in System Design*, 48(3):152–174, 2016.
- [14] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. AFL++ : Combining incremental steps of fuzzing research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, August 2020.
- [15] Patrice Godefroid, Nils Klarlund, and Koushik Sen. Dart: directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '05, page 213–223, New York, NY, USA, 2005. Association for Computing Machinery.
- [16] Jingxuan He, Gishor Sivanrupan, Petar Tsankov, and Martin Vechev. Learning to explore paths for symbolic execution. pages 2526–2540, 11 2021.
- [17] Jie Hu, Yue Duan, and Heng Yin. Marco: A stochastic asynchronous concolic explorer. In *Proceedings of the 46th IEEE/ACM International conference on software engineering*, pages 1–12, 2024.
- [18] Jack Kiefer. Sequential minimax search for a maximum. *Proceedings of the American mathematical society*, 4(3):502–506, 1953.
- [19] James C King. Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394, 1976.

- [20] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *Proceedings of the 45th International Conference on Software Engineering*, ICSE '23, page 919–931. IEEE Press, 2023.
- [21] Yihe Li, Ruijie Meng, and Gregory J. Duck. Large language model powered symbolic execution. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):3148–3176, October 2025.
- [22] Daniel Liew, Daniel Schemmel, Cristian Cadar, Alastair F. Donaldson, Rafael Zahl, and Klaus Wehrle. Floating-point symbolic execution: A case study in n-version programming. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 601–612, 2017.
- [23] Changshu Liu, Alireza Ghazanfari, Yang Chen, and Reyhaneh Jabbarvand. Evaluating code reasoning abilities of large language models under real-world settings. *arXiv preprint arXiv:2512.14917*, 2025.
- [24] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Zhilin Tian, Yuekai Huang, Jun Hu, and Qing Wang. Testing the limits: Unusual text inputs generation for mobile app crash detection with large language model. pages 1–12, 04 2024.
- [25] Zhengxiong Luo, Huan Zhao, Dylan Wolff, Cristian Cadar, and Abhik Roychoudhury. Agentic concolic execution.
- [26] Ardi Madadi, Seongmin Lee, Cornelius Aschermann, and Marcel Böhme. In bugs we trust? on measuring the randomness of a fuzzer benchmarking outcome. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering*, FSE'26, 2026.
- [27] Valentin J. M. Manes, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J. Schwartz, and Maverick Woo. The art, science, and engineering of fuzzing: A survey, 2019.
- [28] Sergey Mechtaev, Alberto Griggio, Alessandro Cimatti, and Abhik Roychoudhury. Symbolic execution with existential second-order constraints. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 389–399, 2018.
- [29] Rui Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. Large language model guided protocol fuzzing. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2024.
- [30] Sun Microsystems. Freely distributable libm (fdlibm). <https://www.netlib.org/fdlibm/>. Reference implementation of elementary mathematical functions.
- [31] OpenAI. Gpt-5 technical overview. <https://openai.com/research>, 2025. Model version: gpt5-2025-08-07.
- [32] Sebastian Poeplau and Aurélien Francillon. Symbolic execution with symcc: don't interpret, compile! In *Proceedings of the 29th USENIX Conference on Security Symposium*, SEC'20, USA, 2020. USENIX Association.
- [33] Saurabh Srivastava, Sumit Gulwani, Swarat Chaudhuri, and Jeffrey S. Foster. Path-based inductive synthesis for program inversion. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '11, page 492–503, New York, NY, USA, 2011. Association for Computing Machinery.
- [34] Nick Stephens, John Grosen, Christopher Salls, Andrew Dutcher, Ruoyu Wang, Jacopo Corbetta, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. Driller: Augmenting fuzzing through selective symbolic execution. In *23rd Annual Network and Distributed System Security Symposium, NDSS 2016*, 23rd Annual Network and Distributed System Security Symposium, NDSS 2016. The Internet Society, 2016. Publisher Copyright: © 2016 Internet Society.; 23rd Annual Network and Distributed System Security Symposium, NDSS 2016 ; Conference date: 21-02-2016 Through 24-02-2016.
- [35] Haoxin Tu, Lingxiao Jiang, and Marcel Böhme. Vital: Vulnerability-oriented symbolic execution via type-unsafe pointer-guided monte carlo tree search, 2025.
- [36] Haoxin Tu, Seongmin Lee, Yuxian Li, Peng Chen, Lingxiao Jiang, and Marcel Böhme. Cottontail: Large language model-driven concolic execution for highly structured test input generation, 2025.
- [37] Willem Visser, Corina S Păsăreanu, and Sarfraz Khurshid. Test input generation with java pathfinder. In *Proceedings of the 2004 ACM SIGSOFT international symposium on Software testing and analysis*, pages 97–107, 2004.
- [38] Yaoxuan Wu, Xiaojie Zhou, Ahmad Humayun, Muhammad Ali Gulzar, and Miryung Kim. Generating and understanding tests via path-aware symbolic execution with llms, 2025.
- [39] Hui Xu, Zirui Zhao, Yangfan Zhou, and Michael R. Lyu. On benchmarking the capability of symbolic execution tools with logic bombs, 2018.
- [40] Insu Yun, Sangho Lee, Meng Xu, Yeongjin Jang, and Taesoo Kim. Qsym: a practical concolic execution engine tailored for hybrid fuzzing. In *Proceedings of the 27th USENIX Conference on Security Symposium*, SEC'18, page 745–761, USA, 2018. USENIX Association.